Лекция 8 Непрерывная интеграция и quality gates

Курс: Управление процессами машинного обучения MLOps Продолжительность: 2 академических часа Связанная работа: ЛР8 «Непрерывная интеграция и quality gates»

Цель лекции

Показать непрерывную интеграцию как проверяемый механизм принятия изменений, а не как набор команд в облачном runner. После занятия студент должен проектировать события, jobs и gates, ограничивать permissions, диагностировать отказ, сохранять evidence и связывать branch protection с конкретным commit.

План занятия

Время	Содержание	Действие преподавателя
0–10 мин	От локального теста к правилу интеграции	Сравнить «у меня прошло» и required check
10–22 мин	Event, workflow, job, step, runner	Разобрать структуру `ci.yml`
22–35 мин	Дизайн quality gates	Расставить проверки по стоимости и риску
35–47 мин	Permissions и недоверенный PR	Показать границу token и опасность контекста
47–58 мин	Concurrency и актуальность run	Связать branch, ref, SHA и отмену
58–70 мин	Artifacts и доказательства	Открыть JUnit, validation и OpenAPI reports
70–82 мин	Диагностика красного run	Найти первый failed step и корневую причину
82–90 мин	Branch protection и ЛР8	Зафиксировать индивидуальный failure/fix

1 Что именно интегрирует CI

Начните с ситуации: два участника по отдельности получили зелёные tests, но объединённый pull request ломает сборку. CI проверяет не намерение автора и не состояние его ноутбука, а конкретный commit в известной среде.

Непрерывная интеграция отвечает на вопросы:

- какое изменение проверяется;
- на каком событии началась проверка;
- какие обязательные контракты применены;
- какие артефакты получены;
- может ли изменение попасть в `main`;
- кто и по какому доказательству диагностировал отказ.

Зелёная иконка без известного SHA и списка gates — слабое доказательство. Run должен относиться к HEAD принимаемого pull request.

2 Локальная проверка и remote check

Локальные команды дают быстрый feedback и помогают не тратить runner time. Remote check выполняется в чистой среде и применяется одинаково к участникам.

Локально	В CI
среда могла быть изменена вручную	runner создаётся заново
доступны незакоммиченные файлы	проверяется commit
могут существовать скрытые caches	cache объявляется явно
автор может пропустить команду	workflow задаёт обязательный набор
результат легко потерять	status и artifact связаны с run

Оба контура нужны. CI не должен быть первым местом, где студент узнаёт о синтаксической ошибке.

3 Модель GitHub Actions

Workflow — YAML-файл автоматизации. Event создаёт run; run содержит jobs; job выполняется на runner; job состоит из steps. Step запускает shell-команду или action.

Проведите чтение сверху вниз:

1. `name` — имя workflow;
2. `on` — события;
3. `permissions` — возможности token;
4. `concurrency` — политика одновременных runs;
5. `jobs` — независимые границы выполнения;
6. `needs` — зависимость jobs;
7. `steps` — фактические команды.

YAML корректен синтаксически — не значит, что процесс корректен инженерно. Workflow может всегда завершаться успешно, проверять не тот каталог или собирать image до tests.

4 События и контекст доверия

Для учебного CI используются:

- `pull_request` — проверка предлагаемого изменения;
- push в `main` — контроль принятой истории;
- `workflow_dispatch` — управляемый ручной запуск.

`pull_request_target` выполняется в контексте базового репозитория и требует особенно осторожного обращения с кодом из fork. Не используйте его как замену `pull_request` для выполнения недоверенного student code.

Попросите студентов назвать, какие значения меняются между событиями: ref, SHA, actor, доступность secrets и права token.

5 Принцип минимальных полномочий

```
permissions:
  contents: read
```

Если workflow только читает код и запускает проверки, write permission не требуется. Checkout выполняется с `persist-credentials: false`, чтобы последующие команды не получили сохранённый Git credential.

Объясните, что permission — часть модели угроз. Скомпрометированная зависимость или ошибочный скрипт ограничены тем, что разрешено token. Нельзя компенсировать неясность выдачей `write-all`.

6 Quality gate как решение о риске

Gate — проверка, неуспех которой блокирует продвижение изменения. Хороший gate:

- контролирует названный риск;
- воспроизводим локально или имеет ясный remote-контур;
- завершает процесс ненулевым exit code;
- выдаёт диагностическое доказательство;
- не зависит от ручного редактирования отчёта;
- имеет владельца и понятный порядок исключения.

В проекте применяются lint, format, data contract, tests, OpenAPI export, smoke training и container build. Не каждая метрика должна сразу стать gate: порог вводится после определения владельца, данных и реакции на отказ.

7 Порядок gates

Сначала выполняют быстрые и дешёвые проверки. Если lint занимает секунды, нет смысла до него строить container несколько минут.

Рекомендуемая последовательность:

1. установка воспроизводимых зависимостей;
2. lint и format check;
3. data contract;
4. unit и contract tests;
5. OpenAPI export;
6. smoke training;
7. container build.

Разделение на jobs `quality` и `container` делает зависимость видимой. `needs: quality` не только оптимизирует время, но и отражает решение: непроверенный код не становится поставляемым образом.

8 Fail fast и полный отчёт

Fail fast уменьшает стоимость: после первой критической ошибки обычные steps не выполняются. Но диагностика уже созданных проверок должна сохраниться.

```
- name: Upload quality evidence
  if: always()
  uses: actions/upload-artifact@v4
```

`always()` не превращает упавший job в успешный. Он разрешает сервисному шагу загрузить доступные reports. Если ранний lint упал до создания каталога, `if-no-files-found: warn` фиксирует отсутствие без подмены причины.

9 Diagnostic artifacts

Лог удобен для последовательности событий; структурированный report — для анализа и сравнения. В ЛР8 сохраняются:

- JUnit XML с test node ID и failure;
- JSON проверки данных;
- OpenAPI JSON принимаемой версии.

Artifact имеет срок хранения. Он не заменяет долговременный registry и не должен содержать `.env`, secrets, персональные данные или полный закрытый dataset.

Попросите открыть artifact и проверить содержимое. Скриншот вкладки Actions не доказывает, что нужный test действительно выполнялся.

10 Concurrency и устаревший результат

```
concurrency:
  group: ${{ github.workflow }}-${{ github.ref }}
  cancel-in-progress: true
```

После нового push предыдущий run той же ветки уже не проверяет HEAD. Его отмена уменьшает очередь. В group входит ref, поэтому runs разных веток не отменяют друг друга.

Отменённый run не равен failed run: он не подтверждает дефект кода. Для индивидуальной сдачи нужен контролируемый отказ конкретного gate.

11 Cache и artifact — не одно и то же

Cache ускоряет повторное получение зависимостей и может быть удалён без потери результата. Artifact сохраняет выход run для человека или следующего процесса.

Нельзя считать cache источником происхождения модели. Для release нужны registry, digest и manifest. Нельзя также кэшировать секреты или полагаться на cache для корректности.

12 Версии Actions

Официальные actions используют major tags, например `actions/checkout@v7`. Major tag удобен для получения совместимых исправлений, но может перемещаться. Полный commit SHA неизменяем и сильнее для supply-chain контроля.

Для учебного шаблона преподаватель проверяет major versions перед семестром. Для критичного production workflow организация может закреплять action по SHA и обновлять его управляемым ботом.

13 Диагностика отказа

На красной странице сначала найдите первый failed step. Последующие steps часто имеют status skipped и не являются причиной.

Алгоритм:

1. проверить event, branch и SHA;
2. отличить failure от cancellation;
3. найти первую неуспешную команду;
4. прочитать exit code и первую содержательную ошибку;
5. открыть report или annotation;
6. воспроизвести команду локально;
7. сформулировать корневую причину;
8. исправить отдельным commit;
9. сопоставить новый run с исправлением.

Кнопка Re-run проверяет нестабильность среды. Она не исправляет детерминированный дефект.

14 Branch protection

CI становится gate только тогда, когда политика ветки требует его успешного завершения. Для `main` нужны pull request, review другого участника, разрешённые обсуждения и required checks `quality` и `container`.

Имена checks должны быть уникальными. Два workflows с одинаковым job name создают неоднозначность. Обход администратором разрушает учебное доказательство, если не оформлен как исключение с причиной.

15 Контролируемый отказ как учебный эксперимент

Каждый студент заранее выбирает один обратимый дефект, прогнозирует failed step, создаёт commit, фиксирует красный run, затем исправляет отдельным commit.

Подходящие варианты: Ruff violation, broken test, invalid data, изменение OpenAPI или ошибка Dockerfile. Ручная отмена, выключение workflow и общий дефект на команду не принимаются.

Эксперимент показывает способность диагностировать систему. Финальный tag содержит только исправленное состояние, но история отказа сохраняется.

16 Типичные антишаблоны

Антишаблон	Последствие	Исправление
`continue-on-error` у обязательного test	красный риск становится зелёным	убрать или оформить явное исключение
secrets доступны PR из fork	утечка полномочий	безопасный event и минимальные permissions
mutable `latest`	нет связи image с commit	SHA/tag/digest
один огромный shell step	плохая диагностика	именованные steps по контрактам
только screenshots	нет машиночитаемого evidence	artifacts и ссылки на run
re-run вместо fix	дефект остаётся	отдельный исправляющий commit

17 Демонстрационный сценарий

1. Открыть workflow и предсказать порядок steps.
2. Показать permission и concurrency group.
3. Ввести безопасный Ruff defect в ветке.
4. Найти failure и annotation.
5. Исправить отдельным commit.
6. Скачать artifact зелёного run.
7. Показать блокировку merge при красном check.
8. Сопоставить финальный tag с SHA успешного run.

18 Вопросы аудитории

1. Почему успешный run старого commit не разрешает merge нового?
2. Чем cancellation отличается от failure?
3. Когда `if: always()` полезен и чего он не меняет?
4. Почему `contents: write` является риском?
5. Чем cache отличается от artifact?
6. Почему container build расположен после tests?

19 Связь с ЛР8 и ЛР9

В ЛР8 студенты превращают проверки в обязательное правило интеграции. В ЛР9 тот же принцип переносится внутрь ML-workflow: стадии получают состояния, параметры, retries и отдельные отчёты. CI проверяет код flow, а оркестратор наблюдает конкретные запуски обработки.

20 Официальные источники

- GitHub workflow syntax: https://docs.github.com/en/actions/reference/workflows-and-actions/workflow-syntax
- GitHub concurrency: https://docs.github.com/en/actions/concepts/workflows-and-actions/concurrency
- GitHub workflow artifacts: https://docs.github.com/en/actions/concepts/workflows-and-actions/workflow-artifacts
- GitHub protected branches: https://docs.github.com/en/repositories/configuring-branches-and-merges-in-your-repository/managing-protected-branches/about-protected-branches
- GitHub secure use reference: https://docs.github.com/en/actions/reference/security/secure-use
- actions/checkout releases: https://github.com/actions/checkout/releases
- actions/setup-python releases: https://github.com/actions/setup-python/releases